5

LECTURE 5

C++ STL std::set
Binary Search Trees

CS24 · Fall 2026 · Diba Mirza

https://join.iclicker.com/BCRL

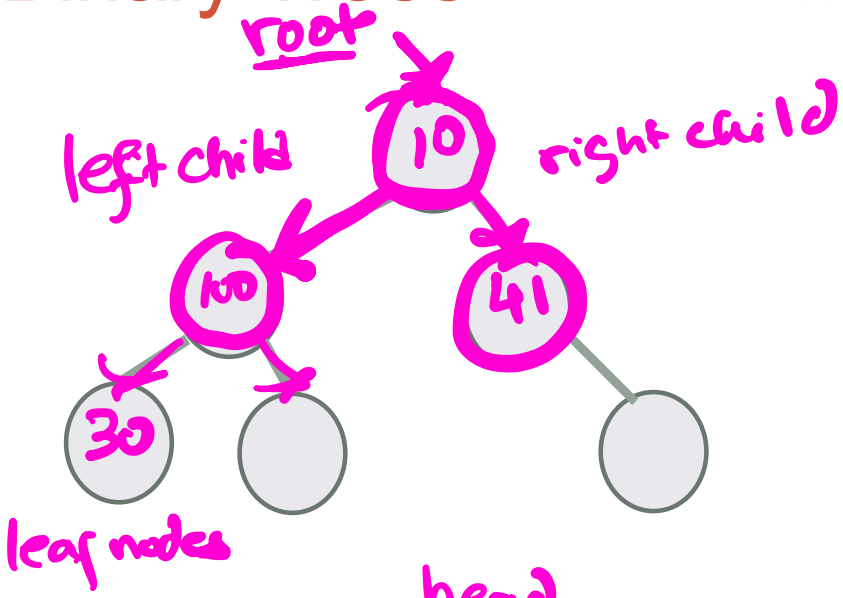
Standard Template Library (STL) TOUR

ADTs in the STL (also called Containers) store a collection of elements

1. `std::vector` — Contiguous, indexable, resizes itself
2. `std::list` — Doubly linked, insert or remove anywhere
3. `std::forward_list` — Singly linked, one direction only
4. `std::set` — Sorted, no duplicates

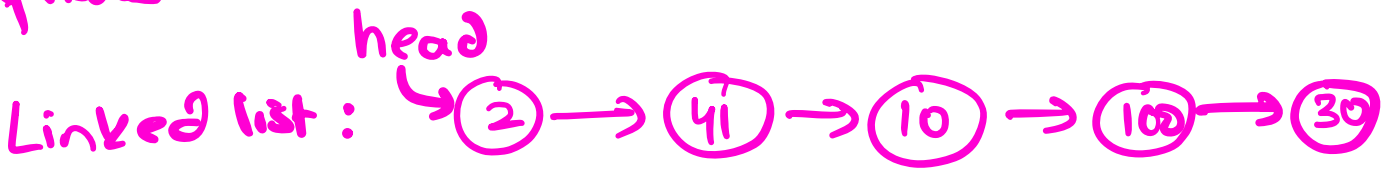
See the docs: https://cplusplus.com/reference/set/set/?kw=set

Binary Trees



In a tree, nodes are arranged in a hierarchy

- One node is distinguished as the root
- Each node:
  - stores a key
  - has a pointer to child nodes and parent (optional)
- Unique path between any two nodes
- Leaf nodes have no children



A set is a binary tree

In a binary tree, each node has at most two children

Which of the following is/are a tree?

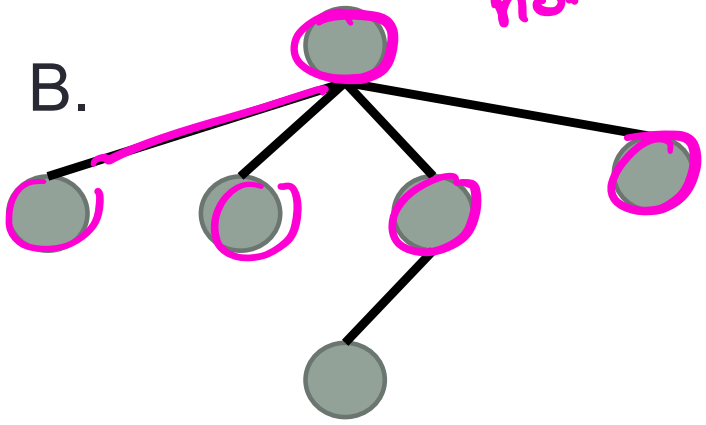
A.



Single node tree

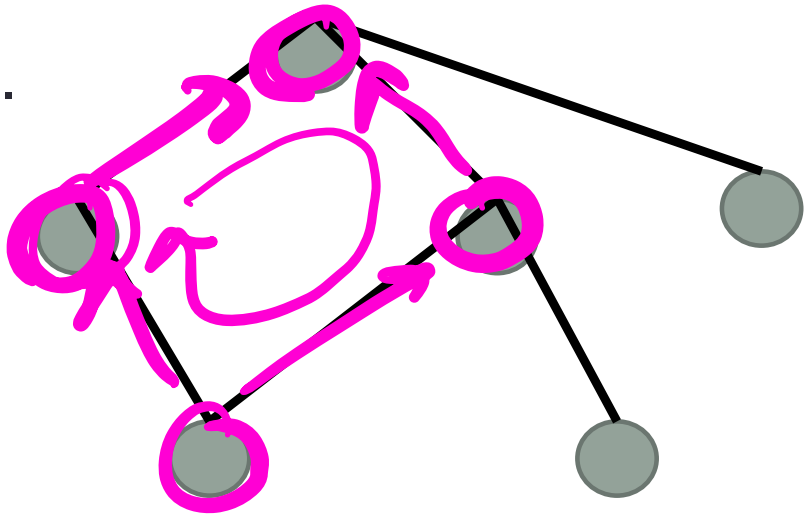
Yes. A is a binary tree

B.



not a binary tree

C.

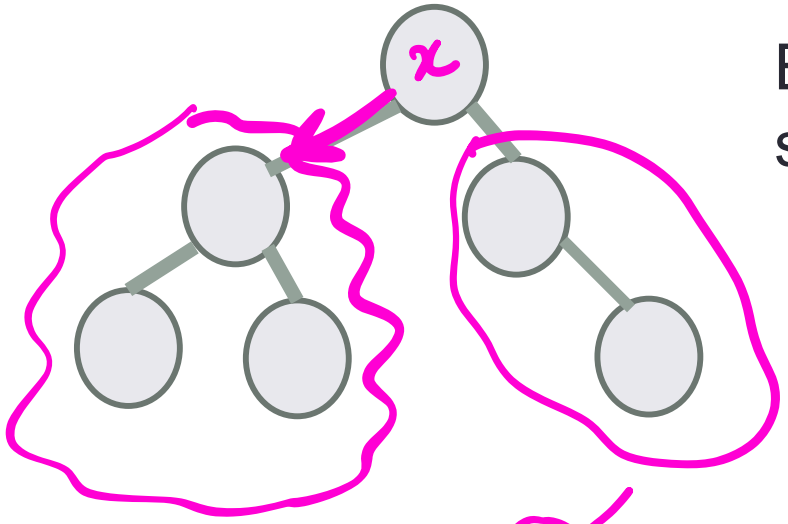


no loops in a tree.

D. A & B

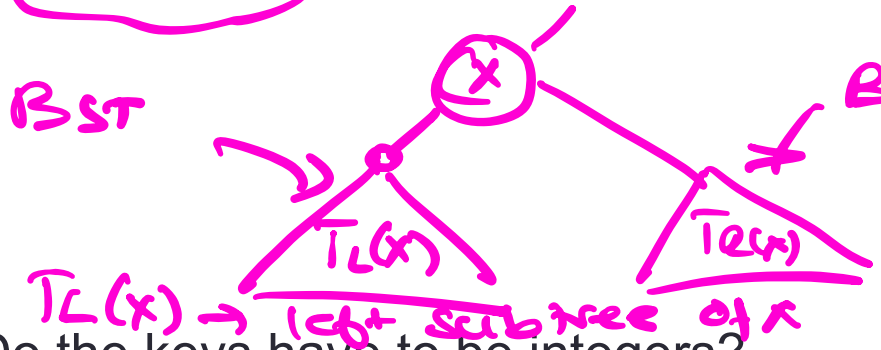
E. All of A-C

Binary Search Tree – What is it?



BST is a binary tree where each node satisfies the Search Tree Property

All key values in the left subtree of x $\leq x \leq$ All key values in the right subtree of x.



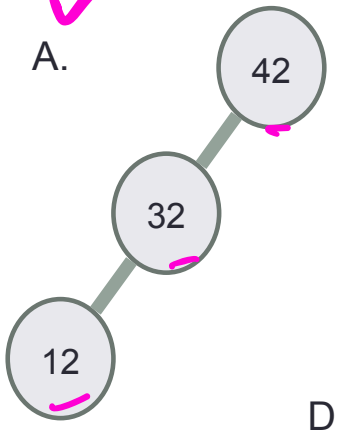
$T_L(x) < x < T_R(x)$

Do the keys have to be integers?

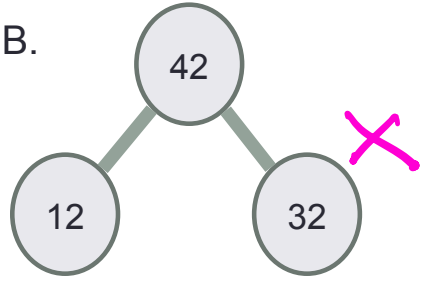
Visualization: https://visualgo.net/en/bst

Which of the following is/are a binary search tree?

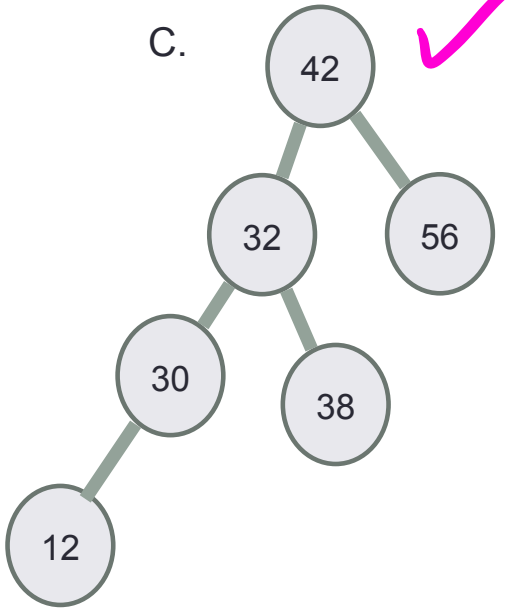
A.



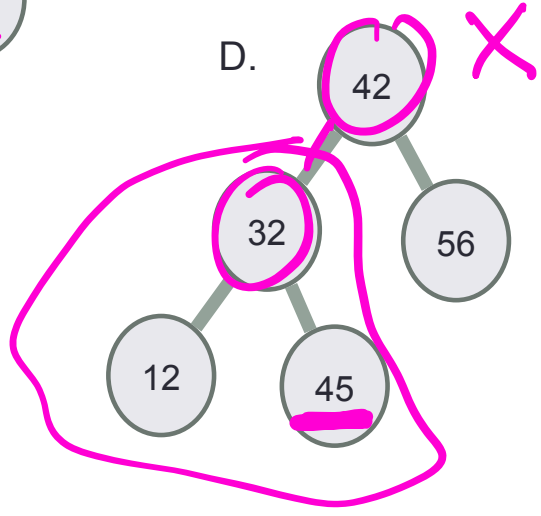
B.



C.



D.



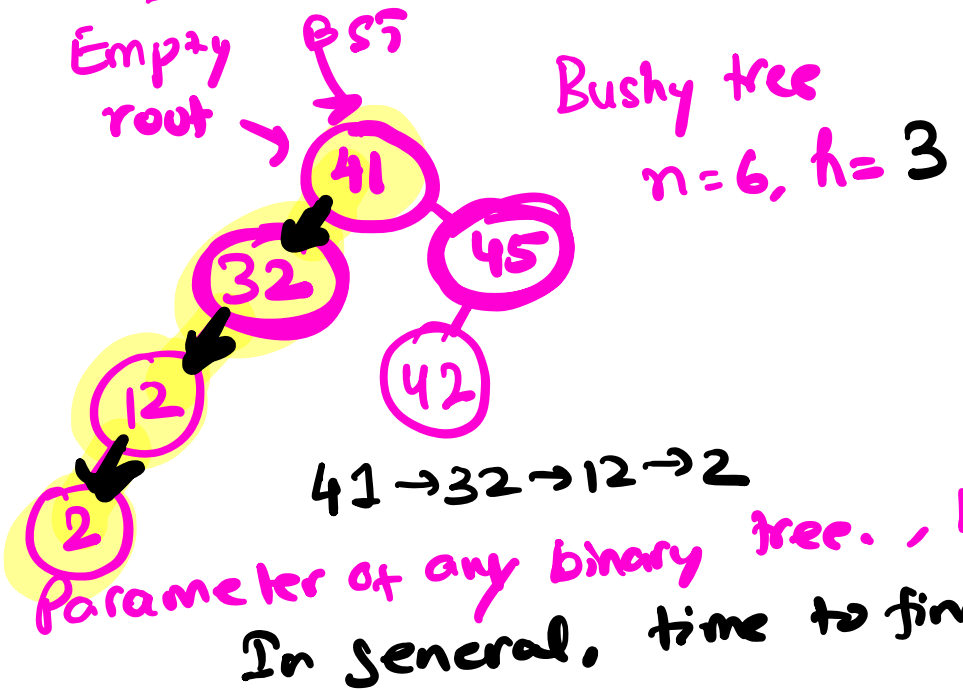
E. More than one of these



Inserting into a BST

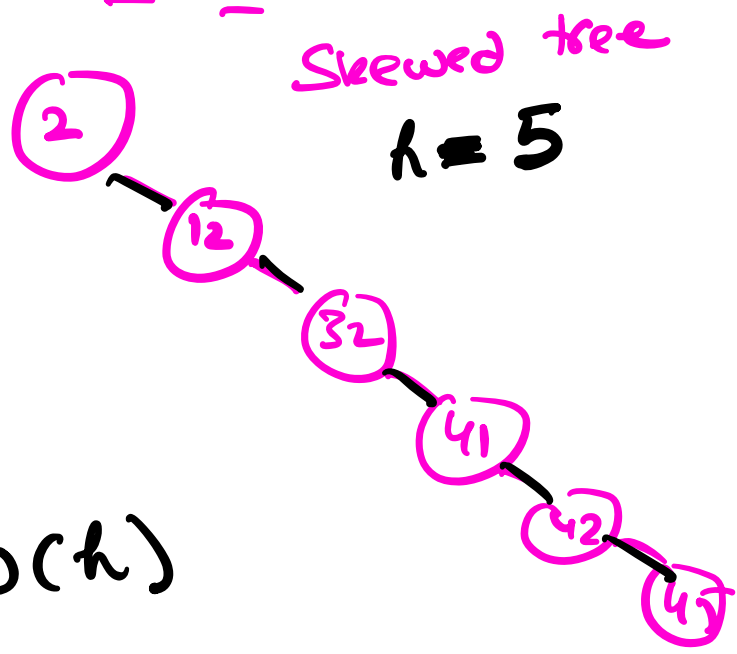
Insert keys:

41, 45, 32, 42, 12, 2



Insert key in sorted order:

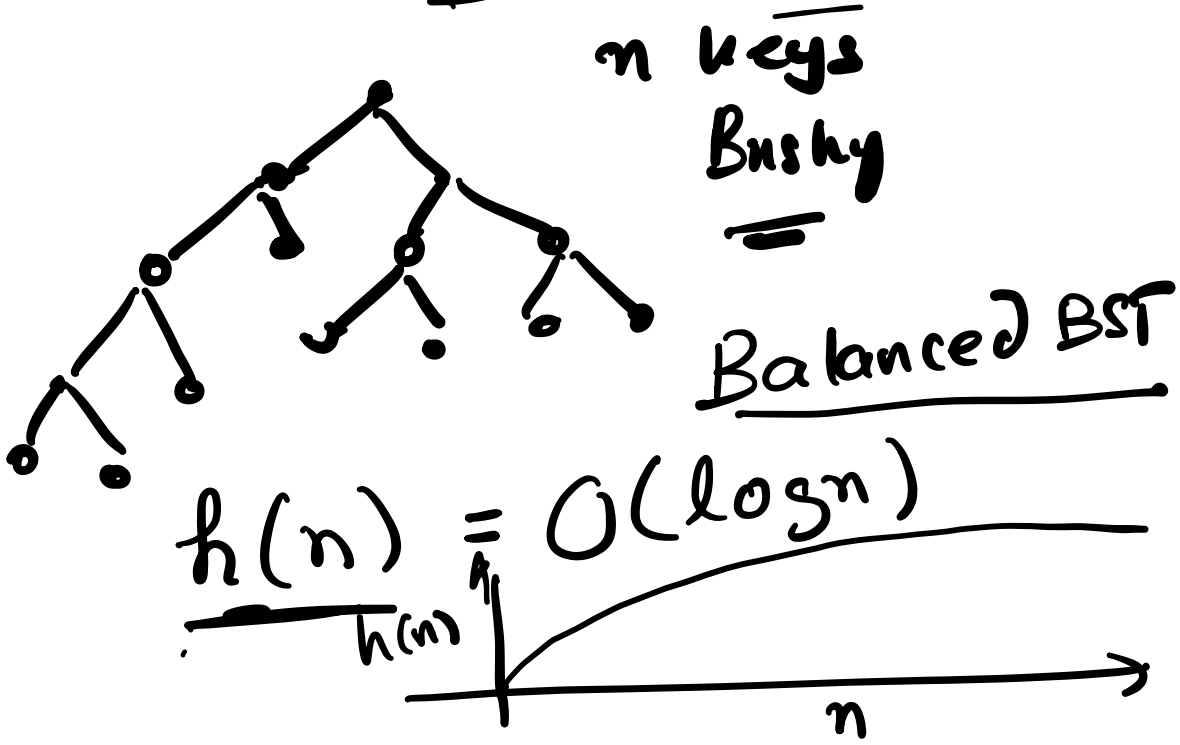
2, 12, 32, 41, 42, 45

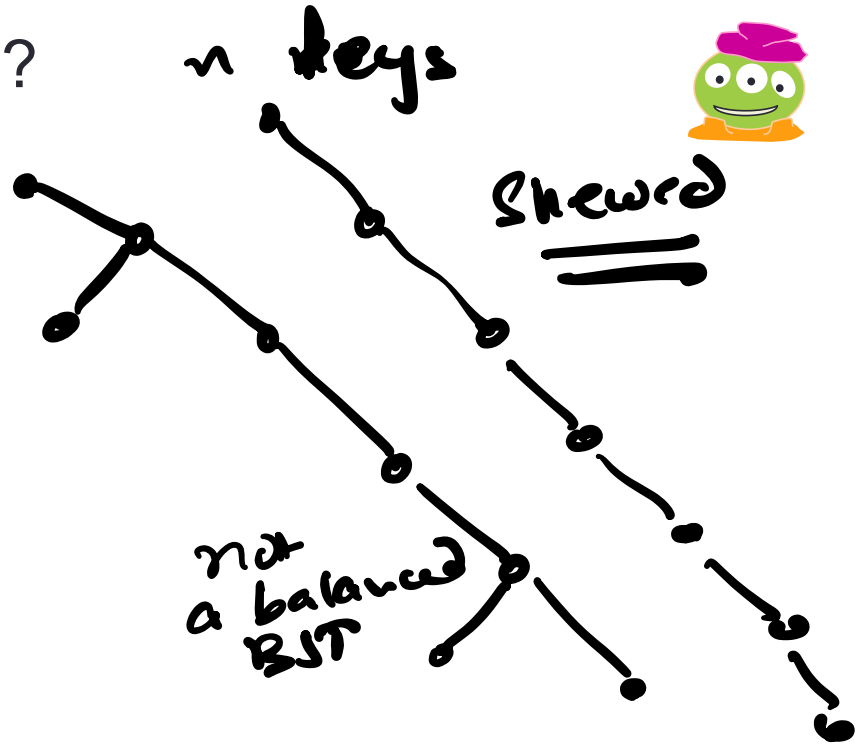


Height of the tree (H): Length of the longest path from the root to a leaf node

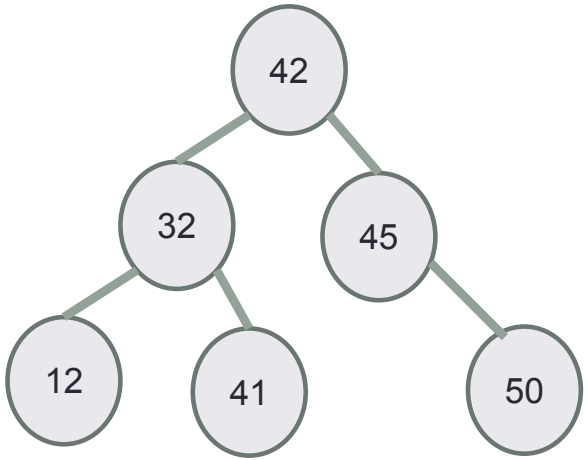
Set is a balanced binary search tree

What is the height of a BST with n nodes?





Search



- Start at the root;
- Trace down a path by comparing **k** with the key of the current node x:
  - If the keys are equal: we have found the key
  - If **k** < key[x] search in the left subtree of x
  - If **k** > key[x] search in the right subtree of x
- What is the running time of search?



Search for 41, then search for 53

What is the running time of search and insert in a BST?

In a BST, both search and insert operations are O(h)

What is the running time, when the tree is skewed h = O(n)

A. O(1)

B. O(logn)

C. O(n)

D. O(nlogn)

What is the running time of search and insert in a BST?

In a BST, both search and insert operations are O(h)

What is the running time, when the tree is balanced h = O(logn)

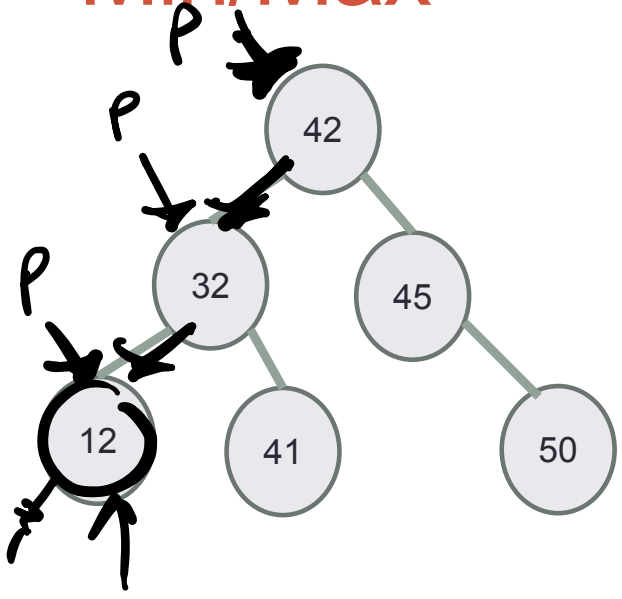
A. O(1)

B. O(logn)

C. O(n)

D. O(nlogn)

Min/Max



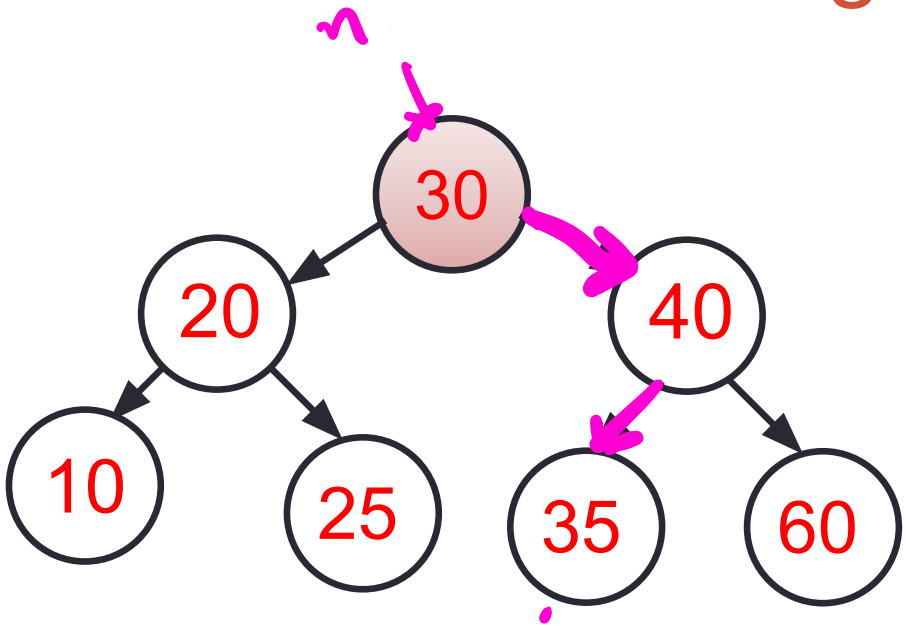
as long as the left child is not null, go to the left child.

Balanced BST. $h = O(\log n)$

min: $O(h) = O(\log n)$

↑ Balanced.

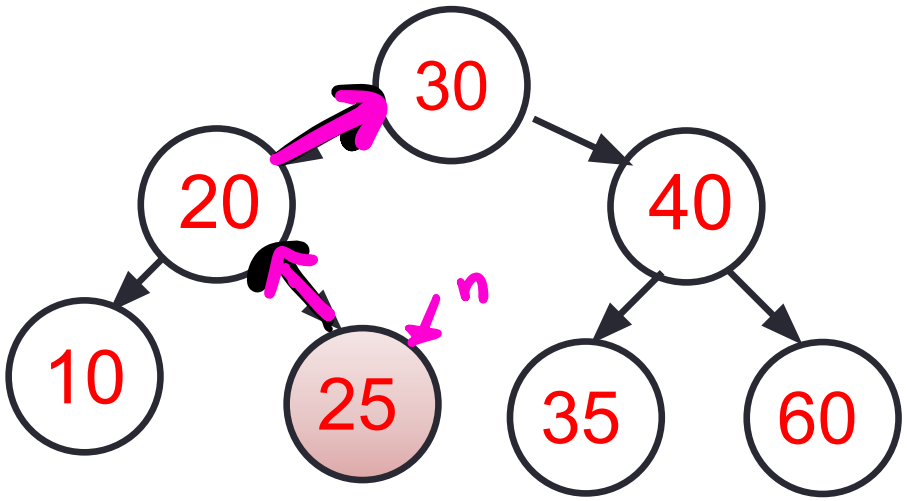
Successor: Next largest element



- Case 1: Node (n) has a right child

return min key in $T_R(n)$

e.g. successor(30) is 35.

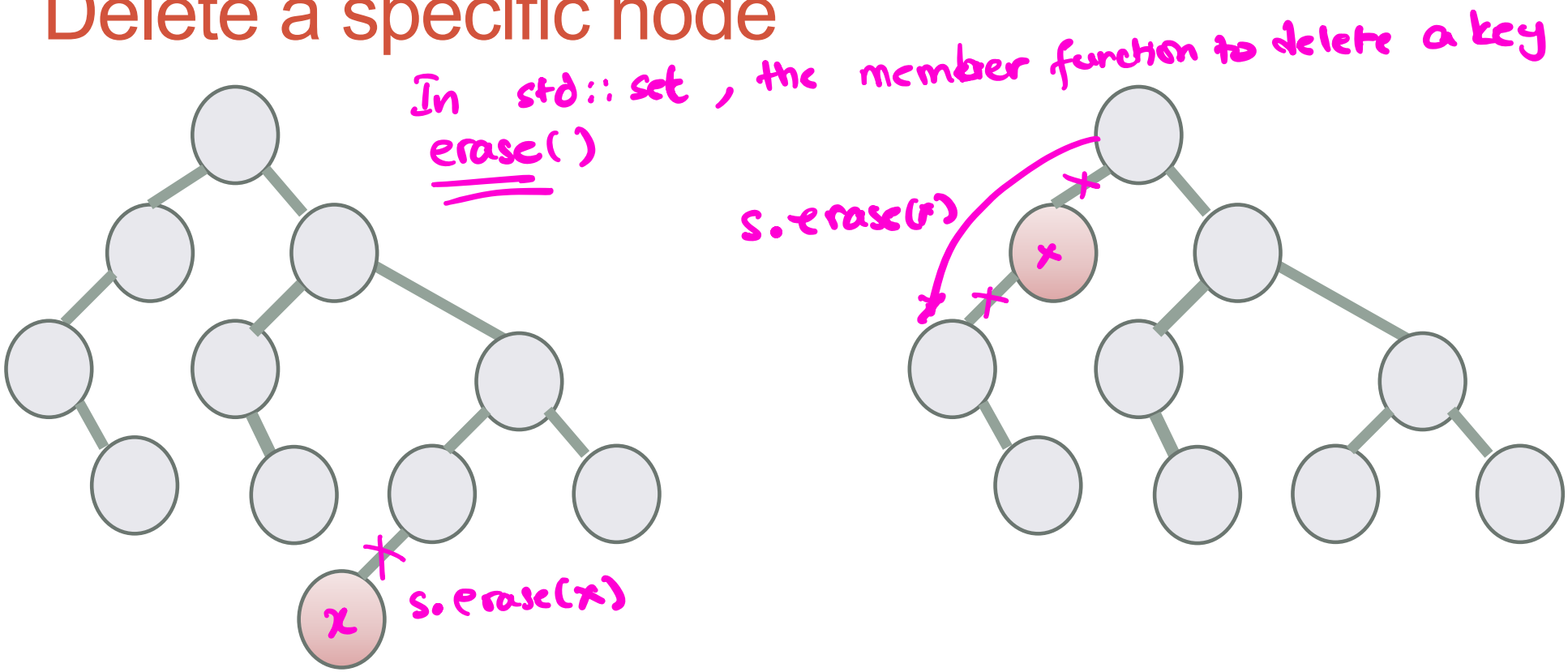


- Case 2: Node (n) has no right child

Traverse up the tree (using parent pointers) until you find a node with key greater than n

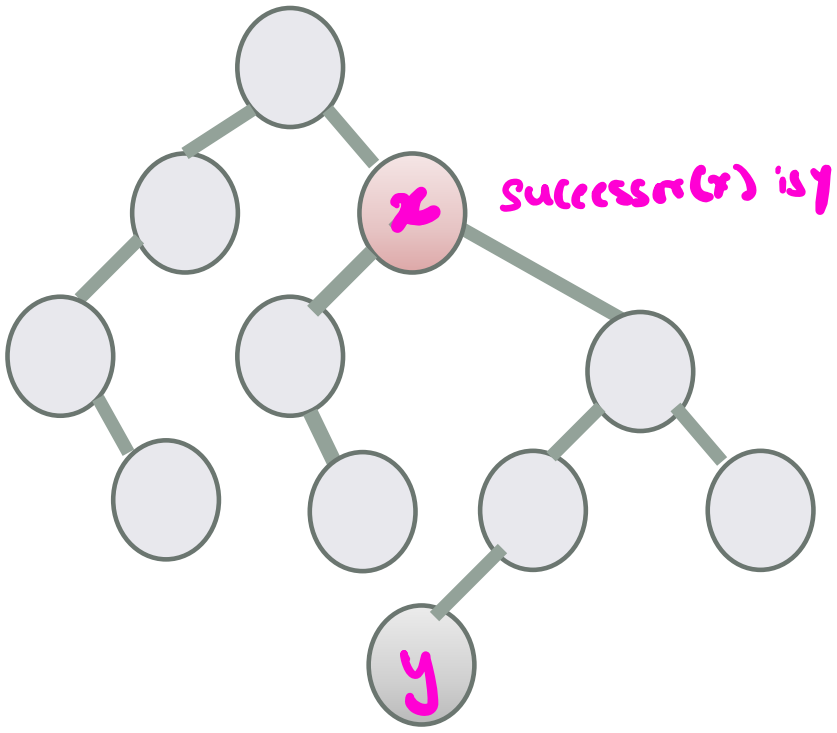
e.g. successor(25) is 30

Delete a specific node



- Case 1: Node is a leaf node
  - Set parent's (left/right) child pointer to null
  - Delete the node
- Case 2: Node has one child (left/right)
  - Replace the node by its only child

Delete a specific node



operation erase(x)

- **Case 3: Node has two children**

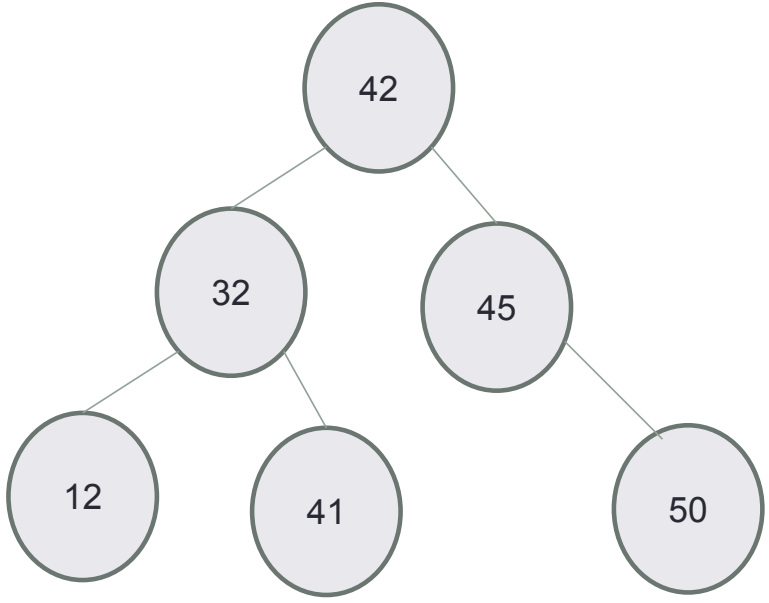
successor(x) is y

Step 1 : swap key of x (x) with successor(x) y
(or predecessor(x))
In the diagram swap x & y
Since x has two children,
(a) successor(x) is necessarily in $T_R(x)$
(b) successor(x) has no left child
Therefore, we can delete x at its new position using case (1) or case (2) of the delete/erase operation

Step 2 : Delete node x at its new position using case 1 or case 2 of delete.

BST ADT

Operations
Search
Insert
Min
Max
Successor
Predecessor
Delete
Print elements In order Preorder, Post order



COMPARE THE CONTAINERS

Big-O cheat sheet -- fill it in

Operation	vector	list	forward_list	set
Index [i]				
Insert *to back*				
Insert *to front*				
Insert *at a position you hold*				
Insert *a value, sorted + unique*				
Find a value				

ANSWER

Big-O cheat sheet

Operation	vector	list	forward_list	set
Index [i]	`O(1)`	n/a	n/a	n/a
Insert *to back*	`O(1)*`	`O(1)`	n/a	n/a
Insert *to front*	`O(n)`	`O(1)`	`O(1)`	n/a
Insert *at a position you hold*	`O(n)`	`O(1)`	`O(1)`	n/a
Insert *a value, sorted + unique*	n/a	n/a	n/a	`O(log n)`
Find a value	`O(n)`	`O(n)`	`O(n)`	`O(log n)`

** push_back is amortized O(1) -- it occasionally resizes and copies everything underneath.*